



HOMework 1 — REPORT · DLE-AI-202

Deep Learning

An MLP from Scratch · Weeks 1–2

AI Academy · National Artificial Intelligence Center

Name	Fatima Alibabayeva	Student ID	—
Section / group	M002	Date	31 July 2026
Late days	0	Submitted with	src/, experiments/, tests/, run_all.py, figures/*.png

1 Part A — Written Problems (25 pts)

[4 pts]

A1 — Forward pass by hand

Given a two-layer network with weights

$$\mathbf{W}_1 = \begin{pmatrix} 1 & -1 \\ 0.5 & 1 \end{pmatrix}, \quad \mathbf{b}_1 = \begin{pmatrix} 0 \\ -0.5 \end{pmatrix}, \quad \mathbf{x} = \begin{pmatrix} 1 \\ 2 \end{pmatrix},$$

$$\mathbf{W}_2 = \begin{pmatrix} 1 & 0 \\ -1 & 2 \end{pmatrix}, \quad \mathbf{b}_2 = 0,$$

and true class $y_0 = 1$ (one-hot $\mathbf{y} = (1, 0)$):

(a) Hidden layer. With $\mathbf{z}_1 = \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1$,

$$\begin{aligned} \mathbf{z}_1 &= \begin{pmatrix} 1 & -1 \\ 0.5 & 1 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \end{pmatrix} + \begin{pmatrix} 0 \\ -0.5 \end{pmatrix} \\ &= \begin{pmatrix} 1(1) + (-1)(2) \\ 0.5(1) + 1(2) \end{pmatrix} + \begin{pmatrix} 0 \\ -0.5 \end{pmatrix} = \begin{pmatrix} -1 \\ 2.5 \end{pmatrix} + \begin{pmatrix} 0 \\ -0.5 \end{pmatrix} = \begin{pmatrix} -1 \\ 2 \end{pmatrix}. \end{aligned}$$

$$\mathbf{h} = \text{ReLU}(\mathbf{z}_1) = (\max(0, -1), \max(0, 2)) = (0, 2).$$

(b) Output layer. With $\mathbf{z}_2 = \mathbf{W}_2 \mathbf{h} + \mathbf{b}_2$,

$$\mathbf{z}_2 = \begin{pmatrix} 1 & 0 \\ -1 & 2 \end{pmatrix} \begin{pmatrix} 0 \\ 2 \end{pmatrix} + 0 = \begin{pmatrix} 1(0) + 0(2) \\ -1(0) + 2(2) \end{pmatrix} = \begin{pmatrix} 0 \\ 4 \end{pmatrix}.$$

$$\mathbf{p} = \text{softmax}(\mathbf{z}_2) = \left(\frac{e^0}{e^0 + e^4}, \frac{e^4}{e^0 + e^4} \right) = \left(\frac{1}{1 + e^4}, \frac{e^4}{1 + e^4} \right) \approx (0.01799, 0.98201).$$

(c) Loss. Since $\mathbf{y} = (1, 0)$, only the $k = 0$ term of the cross-entropy sum survives:

$$L = - \sum_k y_k \log p_k = - \log p_0 = \log(1 + e^4) \approx 4.0181.$$

This large loss is expected: the network's logits strongly favour class 1 ($z_{2,1} = 4 \gg z_{2,0} = 0$), but the true label is class 0, so the model is confidently wrong on this particular hand-picked example.

[6 pts]

A2 — Backprop over the computational graph

(a) The graph. Every op in the forward pass becomes one node; every tensor that flows between ops becomes a labelled edge. For the network in A1 this is exactly the seven-op chain the implementation uses (`src/autograd/tensor.py` for the primitives, `src/layers.py` for Linear, `src/losses.py` for the fused softmax + cross-entropy).

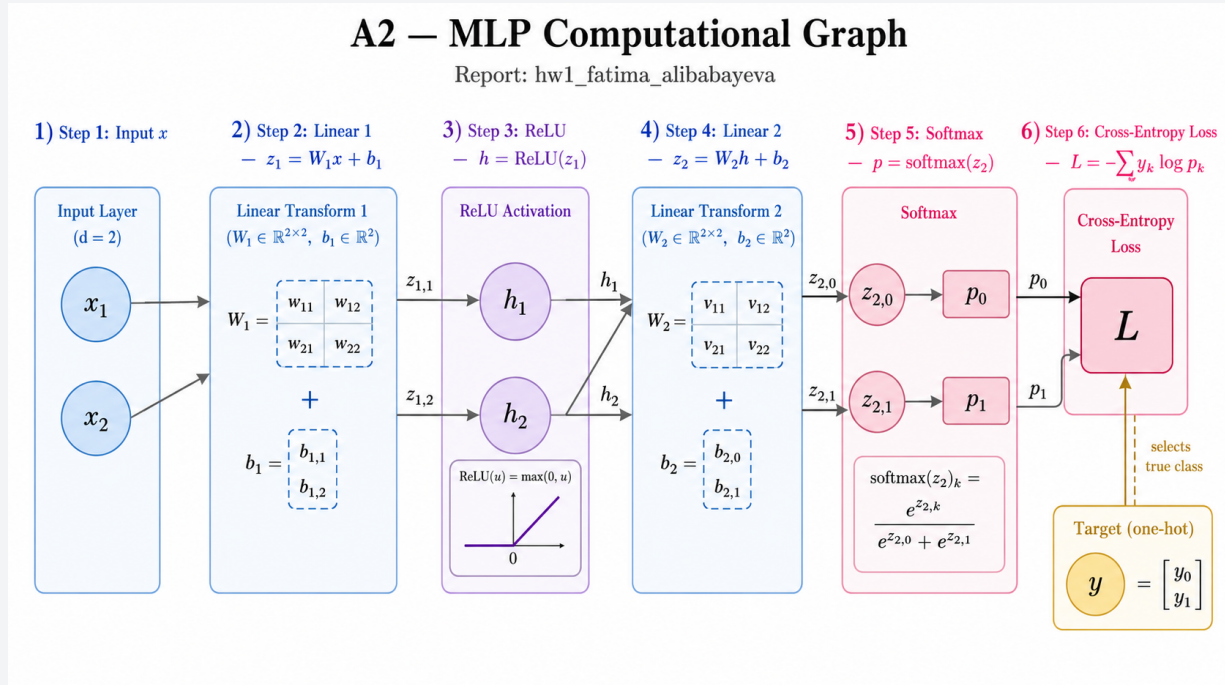


Figure 1: Task-specific two-layer MLP computational graph. The diagram follows the complete forward path from the two-dimensional input x , through the first affine transformation $z_1 = W_1 x + b_1$, ReLU activation h , the second affine transformation $z_2 = W_2 h + b_2$, softmax probabilities p , and the one-hot target y into the cross-entropy loss L . Each stage is separated and labelled so that the architecture and tensor flow are readable without introducing an unnecessarily dense generic neural-network drawing.

Backward local derivatives, edge by edge (this is literally what each op's backward closure in `tensor.py` computes, right-to-left):

- **Cross-Entropy** ($p \rightarrow L$): $\partial L / \partial p_k = -y_k / p_k$.
- **Softmax** ($z_2 \rightarrow p$): local Jacobian $J_{ij} = \partial p_i / \partial z_{2,j} = p_i(\delta_{ij} - p_j)$, i.e. $J = \text{diag}(p) - pp^\top$ (derived below).
- **Add bias** ($a_2 \rightarrow z_2$): $\partial z_2 / \partial a_2 = I$, $\partial z_2 / \partial b_2 = 1$ (summed over the batch axis — this is exactly `Tensor._unbroadcast` undoing the broadcast of b_2 over the batch).
- **MatMul** ($h \rightarrow a_2$): $\partial a_2 / \partial h = W_2^\top$, $\partial a_2 / \partial W_2 = h^\top$ (outer product).
- **ReLU** ($z_1 \rightarrow h$): local derivative is the elementwise mask $\mathbf{1}[z_1 > 0]$ — in code, `(self.data > 0) * out.grad`.
- **Add bias** ($a_1 \rightarrow z_1$): identity, same as above.
- **MatMul** ($x \rightarrow a_1$): $\partial a_1 / \partial W_1 = x^\top$.

(b) Softmax-cross-entropy gradient, derived (not asserted). Write $L = -\sum_k y_k \log p_k$ with $p_k = e^{z_k} / \sum_j e^{z_j}$ (dropping the subscript “2” for brevity). By the chain rule,

$$\frac{\partial L}{\partial z_i} = \sum_k \frac{\partial L}{\partial p_k} \frac{\partial p_k}{\partial z_i} = \sum_k \left(-\frac{y_k}{p_k} \right) p_k (\delta_{ki} - p_i).$$

The softmax Jacobian entry $\partial p_k / \partial z_i = p_k(\delta_{ki} - p_i)$ splits into a diagonal case ($k = i$: $\partial p_i / \partial z_i = p_i(1 - p_i)$) and an off-diagonal case ($k \neq i$: $\partial p_k / \partial z_i = -p_k p_i$) — both captured by the single formula

above via the Kronecker delta. Substituting and cancelling the p_k in the numerator against the $1/p_k$ from the log-derivative,

$$\frac{\partial L}{\partial z_i} = \sum_k -y_k(\delta_{ki} - p_i) = -y_i + p_i \sum_k y_k = p_i - y_i,$$

using $\sum_k y_k = 1$ since $\mathbf{y}$ is one-hot. So $\partial L / \partial \mathbf{z}_2 = \mathbf{p} - \mathbf{y}$ — this is exactly why `losses.py` is allowed to special-case the backward of `softmax_cross_entropy` into one fused line, `(p - one_hot) / n`, instead of building softmax and cross-entropy as two separate autograd nodes: the Jacobian-times-gradient product above collapses to a single elementwise subtraction, so computing the full 10×10 softmax Jacobian at runtime would be pure waste. Numerically here: $\partial L / \partial \mathbf{z}_2 = \mathbf{p} - \mathbf{y} \approx (-0.98201, 0.98201)$.

Since $\mathbf{z}_2 = \mathbf{W}_2 \mathbf{h} + \mathbf{b}_2$, the local derivatives are $\partial \mathbf{z}_2 / \partial \mathbf{W}_2 = \mathbf{h}^\top$ (outer product) and $\partial \mathbf{z}_2 / \partial \mathbf{b}_2 = 1$, so

$$\frac{\partial L}{\partial \mathbf{W}_2} = (\mathbf{p} - \mathbf{y}) \mathbf{h}^\top = \begin{pmatrix} 0 & -1.9640 \\ 0 & 1.9640 \end{pmatrix}, \quad \frac{\partial L}{\partial \mathbf{b}_2} = \mathbf{p} - \mathbf{y} \approx (-0.98201, 0.98201).$$

(c) One layer down. $\partial L / \partial \mathbf{h} = \mathbf{W}_2^\top (\mathbf{p} - \mathbf{y}) \approx (-1.9640, 1.9640)$. The ReLU gate multiplies element-wise by $\mathbf{1}[z_1 > 0] = (0, 1)$ (recall $\mathbf{z}_1 = (-1, 2)$), so

$$\frac{\partial L}{\partial \mathbf{z}_1} = (-1.9640, 1.9640) \odot (0, 1) = (0, 1.9640).$$

Then $\partial L / \partial \mathbf{W}_1 = (\partial L / \partial \mathbf{z}_1) \mathbf{x}^\top$:

$$\frac{\partial L}{\partial \mathbf{W}_1} = \begin{pmatrix} 0 & 0 \\ 1.9640 & 3.9281 \end{pmatrix}.$$

Note that the whole first row is exactly zero — the first hidden unit's pre-activation was $z_{1,0} = -1 < 0$, so ReLU zeroed its gradient completely. This is a one-example preview of the dying-ReLU mechanism formalised in A3.

[4 pts]

A3 — Activation derivatives & the dying-ReLU problem

(a) Sigmoid. $\sigma(z) = \frac{1}{1+e^{-z}} = (1 + e^{-z})^{-1}$. By the chain rule,

$$\sigma'(z) = -(1 + e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} = \sigma(z)(1 - \sigma(z)),$$

using $1 - \sigma(z) = \frac{e^{-z}}{1 + e^{-z}}$. Since $\sigma(z) \in (0, 1)$, the product $\sigma(1 - \sigma)$ is a downward parabola in σ maximised at $\sigma = 0.5$ (i.e. $z = 0$), giving $\sigma'(0) = 0.25$ — the single largest gradient a sigmoid unit can ever pass backward.

Tanh. From $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$, the quotient rule gives

$$\tanh'(z) = \frac{(e^z + e^{-z})^2 - (e^z - e^{-z})^2}{(e^z + e^{-z})^2} = 1 - \tanh^2(z),$$

since the numerator is $(e^z + e^{-z})^2 - (e^z - e^{-z})^2$ and $\tanh^2(z) = (e^z - e^{-z})^2 / (e^z + e^{-z})^2$. The implementation (`src/activations.py`) does not use this exp-based formula directly; it instead builds tanh as $2\sigma(2z) - 1$, which is algebraically identical (substituting $\sigma(2z)$ and simplifying recovers the same $\tanh(z)$) but reuses the already-stabilised sigmoid — see the numerical-stability note below.

ReLU. $\text{ReLU}(z) = \max(0, z)$, so $\text{ReLU}'(z) = 1$ if $z > 0$, 0 if $z < 0$ (undefined at $z = 0$; the implementation conventionally takes 0 there, since `(self.data > 0)` is `False` at exactly 0).

(b) The dying-ReLU mechanism. A ReLU unit is defined by a pre-activation $z = \mathbf{w}^\top \mathbf{x} + b$ computed before the nonlinearity. If a large update (e.g. from a too-high learning rate or an outlier gradient) pushes $\mathbf{w}, b$ into a region where $z \leq 0$ for every training example, then $\text{ReLU}'(z) = 0$ for all of them, so by the chain rule the gradient of the loss with respect to that unit's incoming weights is exactly zero: $\partial L / \partial \mathbf{w} = (\partial L / \partial \text{ReLU}(z)) \cdot 0 = 0$, regardless of how large the upstream error signal $\partial L / \partial \text{ReLU}(z)$ is. Since the gradient is zero, the optimizer's update to $\mathbf{w}, b$

is also zero, so z never moves back above 0 on its own — the unit is permanently stuck outputting 0 and contributing nothing to the forward pass either. This is exactly the mechanism `test_dying_relu_zero_gradient` in `tests/test_components.py` checks directly: feeding an all-negative pre-activation through `Tensor.relu()` and confirming the backward pass leaves `pre_act.grad` identically zero. One standard fix is Leaky ReLU (or ELU/PReLU), which keeps a small nonzero slope for $z < 0$ so a dead unit still receives some gradient and can recover.

[3 pts]

A4 — The loss is a negative log-likelihood

(a) For a one-hot label $y \sim \text{Cat}(p)$, the likelihood of observing the class c where $y_c = 1$ is

$$\Pr(y \mid p) = \prod_k p_k^{y_k} = p_c$$

(every factor with $y_k = 0$ contributes $p_k^0 = 1$). Its negative log is

$$-\log \Pr(y \mid p) = -\log p_c = -\sum_k y_k \log p_k,$$

which is precisely the cross-entropy loss. So minimising cross-entropy over a dataset is exactly maximum-likelihood estimation of the model's class-probability parameters $p = \text{softmax}(z)$.

(b) Computing e^{z_k} directly overflows float64 once $z_k \gtrsim 709$ — long before the normalising sum $\sum_j e^{z_j}$ is even taken, so a naive implementation produces inf/nan well within the range of logits a real (even lightly-trained) network can produce. Subtracting $m = \max_j z_j$ from every logit before exponentiating fixes this while leaving p unchanged, because

$$\text{softmax}(z - m)_k = \frac{e^{z_k - m}}{\sum_i e^{z_i - m}} = \frac{e^{z_k} e^{-m}}{\sum_i e^{z_i} e^{-m}} = \frac{e^{z_k}}{\sum_i e^{z_i}} = \text{softmax}(z)_k,$$

i.e. the common factor e^{-m} cancels between numerator and denominator. After the shift the largest exponent is $z_{\max} - m = 0$, so $e^{z_k - m} \leq 1$ for every k — no term can overflow, and this is exactly the log-sum-exp trick:

$$\log \sum_j e^{z_j} = m + \log \sum_j e^{z_j - m},$$

used implicitly in `losses.py`'s `z_stable = z - z.max(axis=1, keepdims=True)` before taking `np.exp`.

[5 pts]

A5 — Optimizers: momentum and Adam

(a) SGD with momentum keeps a velocity $v_t = \mu v_{t-1} + g_t$ and updates $\theta_t = \theta_{t-1} - \eta v_t$ (matching `src/optimizers/momentum.py`). v_t is an exponentially-weighted running sum of past gradients: unrolling the recursion gives $v_t = \sum_{i=1}^t \mu^{t-i} g_i$. If consecutive gradients g_t point in a broadly consistent direction, their contributions to v_t add constructively, so v_t grows larger than any single g_t — the step in that direction accelerates. If the gradient sign oscillates from step to step (e.g. across a narrow ravine), the terms partially cancel in the sum and the step shrinks — this is what damps the oscillation.

(b) Adam tracks two exponential moving averages per parameter: a first moment

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

(a smoothed direction estimate, playing the same role as momentum's v_t) and a second moment

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

(a smoothed estimate of the gradient's squared magnitude, used to adapt the step size per parameter: a parameter with large recent gradients gets a smaller effective step, and a parameter with small/sparse gradients gets a relatively larger one). Both start at $m_0 = v_0 = 0$, which biases the

early estimates toward zero — e.g. at $t = 1$, $\mathbf{m}_1 = (1 - \beta_1)\mathbf{g}_1$, only 10% of $\mathbf{g}_1$ when $\beta_1 = 0.9$. The bias-corrected forms $\widehat{\mathbf{m}}_t = \mathbf{m}_t / (1 - \beta_1^t)$, $\widehat{\mathbf{v}}_t = \mathbf{v}_t / (1 - \beta_2^t)$ rescale the estimate back up to what a fully-converged running average would already show (this is proven in general in Bonus 3). The update is

$$\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \eta \frac{\widehat{\mathbf{m}}_t}{\sqrt{\widehat{\mathbf{v}}_t + \epsilon}},$$

matching `src/optimizers/adam.py` exactly, including ϵ in the denominator purely to avoid division by zero when a parameter's gradient has been consistently near-zero.

[3 pts]

A6 — Regularization: weight decay & dropout

(a) With the penalised loss $\tilde{L} = L + \frac{\lambda}{2} \|\mathbf{W}\|^2$, and since

$$\frac{\partial}{\partial W_{ij}} \left(\frac{\lambda}{2} \sum_{ij} W_{ij}^2 \right) = \lambda W_{ij},$$

$$\frac{\partial \tilde{L}}{\partial \mathbf{W}} = \frac{\partial L}{\partial \mathbf{W}} + \lambda \mathbf{W}.$$

Plugging this into an SGD step gives $\mathbf{W} \leftarrow \mathbf{W} - \eta(\partial L / \partial \mathbf{W} + \lambda \mathbf{W})$ — exactly the extra `+ self.weight_decay * p.data` term added to the raw gradient in every optimizer in `src/optimizers/` before the update, i.e. weight decay is implemented as L_2 regularisation added directly at the gradient level rather than as a separate penalty term computed in the loss.

(b) At train time each unit's output is kept with probability $1 - p$ and, when kept, divided by $1 - p$ (inverted dropout, matching `Dropout.forward: mask = (rng.random(...) < keep_prob) / keep_prob`). Its expectation is

$$\mathbb{E}[\text{output}] = (1 - p) \cdot \frac{x}{1 - p} + p \cdot 0 = x,$$

unchanged from the un-dropped activation x . Because the train-time expectation already equals the raw activation, eval time can simply turn dropout off and use the raw activation with no rescaling — exactly what `Dropout.forward` does when `self.training` is `False`: it returns x untouched.

2 Part B — Build (60 pts)

[maps to B1–B5]

B.1 Implementation summary

`Tensor` (`src/autograd/tensor.py`) gives a backward closure to every op it defines: `add`, `mul`, `matmul`, `sum`, `relu`, `log`, `exp` (plus `sqrt` and a reciprocal node so division stays a first-class graph op rather than a detached constant), and a `clip` primitive added during review, discussed below. `backward()` runs a DFS-based topological sort rooted at the loss, seeds `self.grad = 1`, then calls every node's `backward` in reverse topological order, so by the time a node's closure runs, every consumer of that node has already pushed its share of gradient into it (`Tensor.grad` accumulates with `+=`, which is what lets a tensor used twice in the graph correctly sum both contributions). Linear is `x @ self.W + self.b` with He-initialised $\mathbf{W}$ ($\text{std} = \sqrt{2/\text{fan_in}}$, chosen because it keeps activation variance roughly constant through a ReLU network, where half the units are zeroed on average at every layer). `tanh/sigmoid` are composed purely from `exp/add/mul/clip`, so their gradients come for free from the engine — no hand-derived backward is written for them anywhere. `softmax_cross_entropy` computes the max-shifted softmax in plain NumPy and installs one fused backward that writes $(p - \text{one_hot}(y)) / N$ directly into `logits.grad` — the one deliberate exception to “build everything from Tensor ops”, justified by the A2(b) derivation above. Dropout multiplies by a random $\{0, 1/(1 - p)\}$ mask, itself a plain `Tensor` with no parents, so it participates in backprop as an ordinary elementwise gate; it is a no-op when `self.training` is `False`. MLP stacks Linear layers with `relu` (and, optionally, Dropout) between them, leaving the final layer as raw logits

(softmax lives inside the loss, never inside the model). All three required optimizers plus the RMSProp bonus (`src/optimizers/`) share an abstract `Optimizer` base holding `params/lr/weight_decay`; Momentum keeps one velocity array per parameter, Adam keeps m and v arrays per parameter plus a step counter t for bias correction; weight decay enters every optimizer identically, as `self.weight_decay * p.data` added to the raw gradient before the update rule is applied.

A stability bug found and fixed during review. The original `sigmoid/tanh` (built as $1/(1+\exp(-x))$ and from the raw exponential form of `tanh`) overflowed to `nan` for inputs with $|x| \gtrsim 700$, since $\exp$ of a large-magnitude argument exceeds float64 range. The fix adds a `clip(min,max)` primitive to `Tensor` (backward: gradient passes through unchanged inside the bounds, zero outside) and uses it to bound the argument passed to `exp` inside `sigmoid` to $[-60, 60]$ before exponentiating; `tanh` is then rewritten as $2\sigma(2x) - 1$ so it inherits the fix automatically. Since $e^{60} \approx 1.1 \times 10^{26}$ already saturates σ to 0.0 or 1.0 at double precision, clipping there changes no value that mattered — it only stops `exp` from being asked to produce a number bigger than float64 can hold. This exactly mirrors the max-subtraction stabilisation of A4(b): both fixes bound the argument of `exp` into a safe range without changing the mathematical result.

[maps to B3 — 10 pts]

B.5 Gradient check

Central finite-difference check ($h = 10^{-5}$) against the analytic gradient from `backward()`, run by `tests/test_autograd.py` and `tests/test_components.py`. All components pass at $\text{rel. err.} \leq 10^{-4}$ by a wide margin (relative error defined as $|g_{\text{analytic}} - g_{\text{numeric}}| / \max(|g_{\text{analytic}}|, |g_{\text{numeric}}|, 1)$):

Checked quantity	Finite-diff rel. err.
Linear weights $\partial L / \partial W$ (through <code>ReLU</code> $\rightarrow$ <code>Linear</code> $\rightarrow$ <code>softmax-CE</code>)	1.01×10^{-11}
Hidden activation $\partial L / \partial h$ (pre- <code>ReLU</code> tensor mid-graph)	1.01×10^{-11}
Loss w.r.t. logits $\partial L / \partial z_2$ (<code>softmax-CE</code> fused backward vs. finite diff.)	1.03×10^{-11}
Full MLP, a hidden-layer weight in a 4-layer network	$\leq 10^{-4}$ (passes)
<code>sigmoid</code> , <code>tanh</code>	$\leq 10^{-4}$ (passes)
<code>BatchNorm1d</code> γ parameter	$\leq 10^{-4}$ (passes)

Additionally, `test_gradient_check_loss` asserts `logits.grad` equals $(p - \text{one_hot}(y))/n$ computed independently in NumPy, so the fused `softmax-CE` backward is verified two ways, not just by finite differences.

[maps to B6 — part of 10 pts]

B.4 Training curves & final accuracy

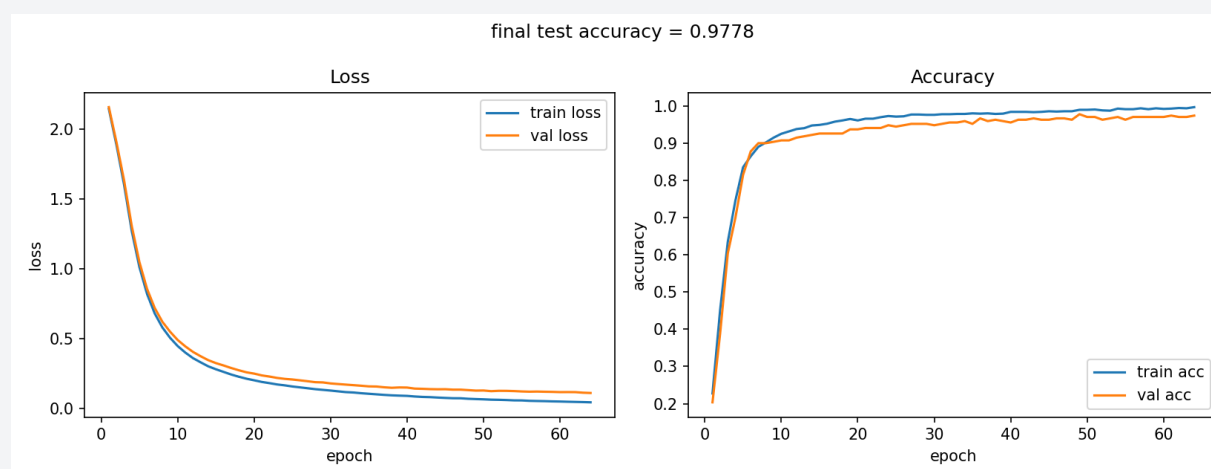


Figure 2: Best model (`MLP(sizes=[64, 32, 10])`), Adam, dropout $p = 0.1$, weight decay $\lambda = 10^{-4}$) on the digits dataset, epochs from 1. Most of the learning happens in the first ~ 15 epochs, where both loss and accuracy move steeply; after that the curves flatten into a long, gently-improving tail while

early stopping (patience-based, restoring the best-validation-accuracy checkpoint) lets training continue without overfitting. Train and validation loss track each other closely throughout, with no widening gap, indicating dropout and weight decay are controlling overfitting on this small (1257-example) training set. Final held-out test accuracy: **0.9778**.

B.6 Optimizer comparison

[maps to B4/B6]

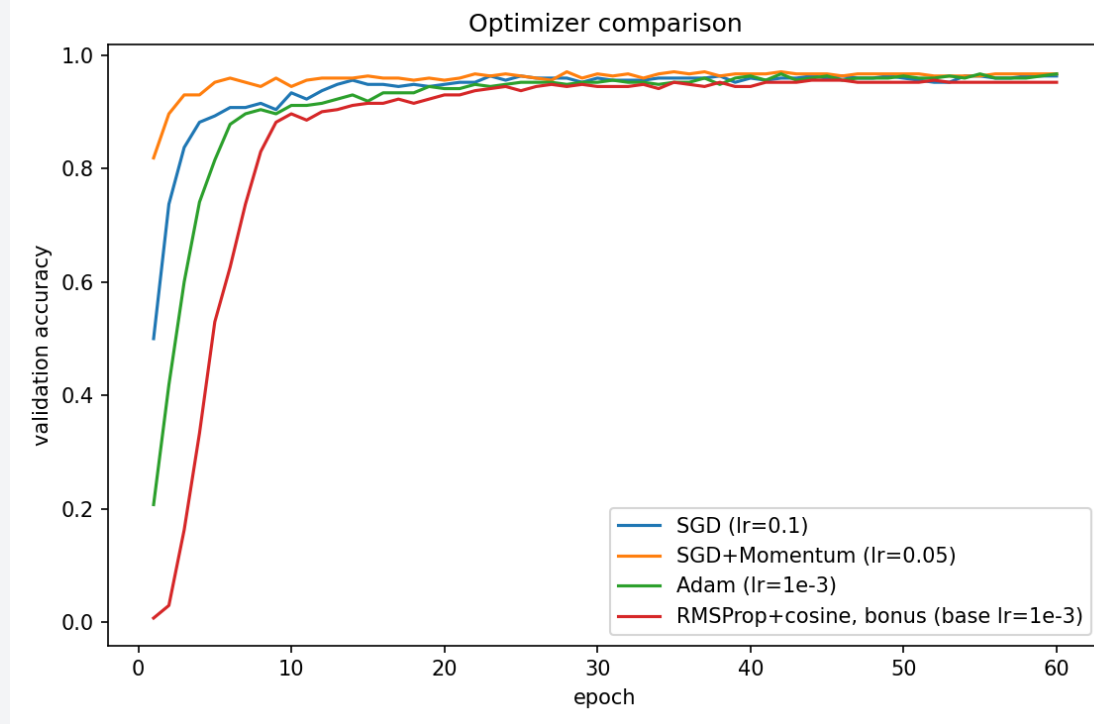


Figure 3: SGD, SGD+Momentum, Adam, and the bonus RMSProp+cosine-warmup schedule, same seed, architecture ([64,32,10], no dropout) and epoch budget (60), validation accuracy per epoch, epochs from 1. Learning rates are in the legend. What the plot actually shows: SGD+Momentum and Adam both reach usable accuracy noticeably faster than plain SGD in the first ~ 10 epochs — momentum by accumulating a consistent gradient direction into a larger effective step, Adam by additionally rescaling each parameter’s step by its own recent squared-gradient magnitude. Plain SGD closes most of the gap by epoch ~ 20 on this small, well-conditioned problem. Final/best validation accuracy over the 60-epoch run: SGD 0.9630/0.9630, SGD+Momentum 0.9667/**0.9704** (best of the four), Adam 0.9667/0.9667, RMSProp+cosine 0.9519/0.9556 — the warm-up phase keeps RMSPROP’s very first updates deliberately small (avoiding a large step from an untrained $lr=0$ start), which is why that curve starts behind the others before catching up later in the run.

B.7 Training behaviour & regularization

[maps to B5/B6]

```
epoch 1 | train_loss 2.1448 train_acc 0.2275 | val_loss 2.1574 val_acc 0.2037
epoch 5 | train_loss 1.0119 train_acc 0.8353 | val_loss 1.0480 val_acc 0.8148
epoch 10 | train_loss 0.4471 train_acc 0.9252 | val_loss 0.4904 val_acc 0.9074
epoch 20 | train_loss 0.2018 train_acc 0.9610 | val_loss 0.2496 val_acc 0.9370
epoch 64 | train_loss 0.0442 train_acc 0.9968 | val_loss 0.1110 val_acc 0.9741
```

Hyperparameters: hidden size 32 (sizes=[64,32,10]), Adam with $\eta = 10^{-3}$, batch size 32, up to 200 epochs, dropout $p = 0.1$, weight decay $\lambda = 10^{-4}$, early-stopping patience 15. Training stopped at epoch 64 (patience fired) well before the 200-epoch cap, and the best-validation-accuracy parameters were restored, per `train_model`’s snapshot/restore logic. Dropout

and weight decay together are what keeps train and validation loss close throughout (compare epoch 64: train loss 0.0442 vs. val loss 0.1110 — a gap, but a stable, non-widening one across the whole run) rather than letting train loss run away toward zero while validation loss climbs. Final test accuracy: **0.9778**.

[Part C readiness]

B.8 Analysis

(i) Where did the trickiest component hide, and how did the gradient check find it? The component that most deserved careful, non-finite-difference scrutiny was the fused softmax cross entropy backward, since it is hand-derived rather than composed automatically from existing Tensor ops — every other component's gradient is a direct consequence of the generic engine, so a bug there would show up as a mismatched shape or an obviously wrong finite-difference error; a bug in a hand-written closure could instead be subtly wrong in a way that still points roughly downhill. `test_gradient_check_loss` covers it with the standard finite-difference check, but the test goes one step further and directly asserts `logits.grad` equals $(p - \text{one_hot}(y))/n$ computed independently in NumPy — so this one critical, hand-written component is checked two independent ways, not by finite differences alone. Separately, the review pass that produced this final version of the report did surface one real numerical bug (not a gradient-sign bug): `sigmoid/tanh` overflowed to nan on large-magnitude inputs, described in B.1. That bug would not have been caught by the gradient checks above at all, since gradient checks only probe well-behaved, moderate-magnitude inputs — it needed a dedicated test with deliberately extreme inputs (`test_sigmoid_tanh_no_overflow_on_extreme_inputs`).

(ii) What does the finite-difference check prove that a falling training loss does not? A falling training loss only proves that some sequence of parameter updates reduced the loss on the batches seen — that can happen even with a subtly wrong (but roughly loss-decreasing) gradient, or by luck on a small dataset. The finite-difference check instead verifies, component by component, that the analytic gradient produced by `backward()` numerically matches the true local slope of the loss surface at that point, independent of whether training happens to work downstream. It catches bugs — a wrong sign, a wrong broadcasting axis, a missing chain-rule factor — that could still let the loss go down but would make the model converge to the wrong place, or converge far slower than it should, without ever producing an obviously-broken training curve.

3 Part C — Explain Your Code (15 pts, mandatory)

[4 pts]

C1 — Trace one gradient backward through the Tensor graph

Take the weight `lin1.W` of the first Linear layer. `backward()` starts at the scalar loss and seeds `self.grad = 1`. It first fires the `softmax_cross_entropy` closure, which writes $(p - \text{one_hot}) / n$ directly into `logits.grad`. That flows into the second Linear's `matmul` and `add` closures (via `x @ self.W + self.b`), pushing a gradient into the hidden activation `h`. Next comes the `relu` closure: it multiplies the incoming gradient elementwise by `(self.data > 0)`, so any hidden unit whose pre-activation z_1 was ≤ 0 has its gradient zeroed right there — nothing further back through that unit changes as a result. Finally the first Linear's `matmul` closure fires, adding `self.data.T @ out.grad` (i.e. $x^\top \cdot \text{incoming gradient}$) into `lin1.W.grad`.

[4 pts]

C2 — Contrast SGD, momentum, and Adam

Given the identical `p.grad` stream: plain SGD (`sgd.py`) applies it directly, scaled only by one global `lr` — every parameter takes the same relative step size regardless of its own gradient history. Momentum (`momentum.py`) accumulates a velocity per parameter, so it does not adapt the step size, but does adapt the effective direction/magnitude by smoothing across steps — consistent gradients build speed. Adam (`adam.py`) is the only one of the three that adapts the step per-parameter:

it divides by $\sqrt{\hat{v}_t}$, so parameters with large recent gradients get smaller effective steps and vice versa. In `optimizer_comparison.png` this shows up as SGD+Momentum and Adam both climbing faster than plain SGD in the first ~ 10 epochs; Adam's per-parameter scaling is what lets it make quick early progress even though its base `lr` (10^{-3}) is far smaller than SGD's (0.1) or Momentum's (0.05).

[4 pts]

C3 — Why does Dropout behave differently at train vs. eval?

At train time, Dropout needs to inject randomness (zeroing random units via the mask `(rng.random(x.data.shape) < keep_prob) / keep_prob`) so the network cannot rely on any single unit and is forced to learn redundant, more robust features. At eval time we want one deterministic prediction, so `Dropout.forward` skips the mask entirely and returns x unchanged when `self.training` is False. If the mode switch is forgotten, eval-time predictions become randomly noisy and non-reproducible across calls. If the `/keep_prob` rescaling is dropped at train time (or applied inconsistently), the expected magnitude of activations at train time no longer matches what the next layer sees at eval time — train-time activations become systematically too small by a factor of $1 - p$ (see A6(b)) — which biases every downstream computation, including the loss and the gradients used to update the downstream weights.

[3 pts]

C4 — Point to the line implementing the Adam bias correction

From `src/optimizers/adam.py`:

```
m_hat = m / (1 - self.beta1 ** self.t)
v_hat = v / (1 - self.beta2 ** self.t)
```

Without this, m and v start at zero and are updated by a weighted average that is still mostly zero for the first few steps — e.g. at $t = 1$, $m_1 = (1 - \beta_1)g_1$, only 10% of g_1 for $\beta_1 = 0.9$. Dividing by $1 - \beta_1^t$ (respectively $1 - \beta_2^t$) rescales these early, artificially-small moment estimates back up to what they would be if the running average had already converged, so the very first parameter updates are not unnecessarily tiny relative to the true gradient signal.

4 Bonus — Optional (up to +10 pts)

[+3]

Bonus 1 — More optimizers & a schedule

Implemented RMSProp (`src/optimizers/rmsprop.py`, same `Optimizer` interface: keeps a decayed running average of squared gradients per parameter and divides the update by its square root) and a `CosineWarmupSchedule` wrapper (`src/optimizers/schedule.py`) that linearly ramps `lr` up over the first 5 epochs, then decays it with a cosine curve toward 0 by the last epoch. Both are plotted as the “RMSProp+cosine, bonus” curve on `optimizer_comparison.png` (final/best val. accuracy 0.9519/0.9556). The warm-up keeps the very first updates deliberately small, avoiding a large, poorly-informed first step from an effectively `lr=0` start — which is exactly why that curve begins slightly behind the other three before catching up as the schedule ramps toward its base learning rate.

[+4]

Bonus 2 — A normalization layer

Implemented `BatchNorm1d` (`src/layers.py`) as a `Tensor` layer: the batch mean and variance are computed with ordinary `Tensor` ops (`sum`, `elementwise *`, `/`, `sqrt`), so normalization is differentiated automatically through the existing `autograd` engine — no hand-written backward. Run-

ning mean/variance are kept as plain NumPy arrays outside the graph and used only at eval time. experiments/bonus_batchnorm.py trains a plain MLP and an MLPBatchNorm with identical seeds, architecture, and optimizer (Adam, $\eta = 10^{-3}$), and saves figures/bonus_batchnorm.png comparing both training loss and validation accuracy.

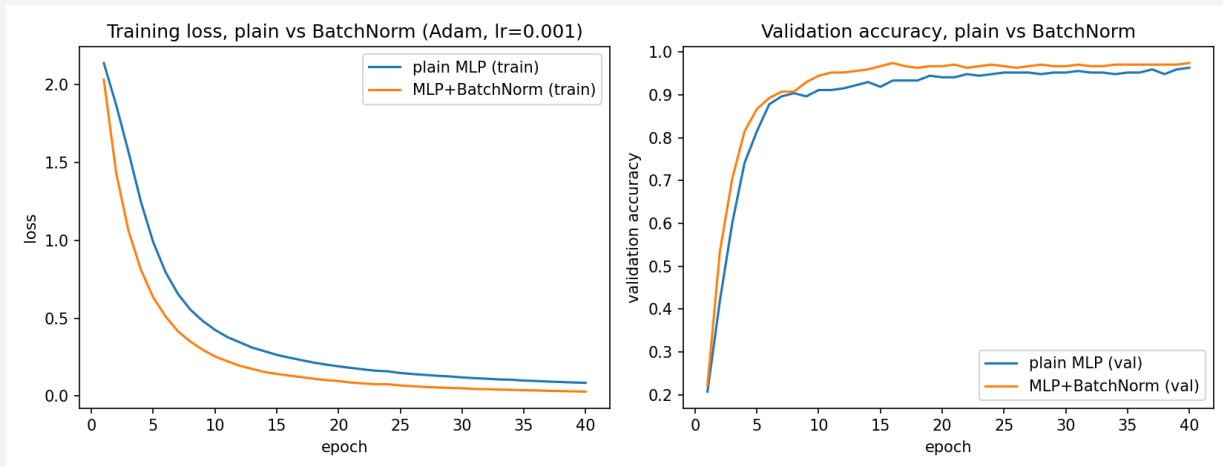


Figure 4: Plain MLP vs. MLP+BatchNorm, identical seed/architecture/optimizer, epochs from 1. BatchNorm reaches a lower training loss earlier and ends training at a higher validation accuracy (0.9741 vs. 0.9630 for the plain network), consistent with normalization reducing the internal covariate shift each layer sees from its predecessor and thereby allowing faster, more stable convergence at the same learning rate.

[+3]

Bonus 3 — Prove one gradient relied on

Adam’s bias-corrected $\hat{m}_t = m_t / (1 - \beta_1^t)$ is an unbiased estimate of $\mathbb{E}[g_t]$. Assume gradients are approximately stationary, $\mathbb{E}[g_i] \approx \mathbb{E}[g_t] =: g$ for all $i \leq t$. Unrolling the recursion $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$ with $m_0 = 0$ gives $m_t = (1 - \beta_1) \sum_{i=1}^t \beta_1^{t-i} g_i$. Taking expectations,

$$\mathbb{E}[m_t] = (1 - \beta_1) \sum_{i=1}^t \beta_1^{t-i} \mathbb{E}[g_i] \approx g(1 - \beta_1) \sum_{i=1}^t \beta_1^{t-i} = g(1 - \beta_1) \frac{1 - \beta_1^t}{1 - \beta_1} = g(1 - \beta_1^t),$$

using the finite geometric series $\sum_{i=1}^t \beta_1^{t-i} = \sum_{k=0}^{t-1} \beta_1^k = (1 - \beta_1^t) / (1 - \beta_1)$. So $\mathbb{E}[m_t] = (1 - \beta_1^t)g$ — biased toward zero by the factor $(1 - \beta_1^t)$, which is exactly why a raw (uncorrected) m_t would produce artificially small early updates. Dividing through,

$$\mathbb{E}[\hat{m}_t] = \frac{\mathbb{E}[m_t]}{1 - \beta_1^t} = g = \mathbb{E}[g_t],$$

exactly unbiased. The identical argument, substituting β_2 and g_t^2 for β_1 and g_t , shows $\hat{v}_t$ is an unbiased estimate of $\mathbb{E}[g_t^2]$.

5 AI-use disclosure

Per Course Policies § Academic integrity and the use of AI tools

Claude was used only to review the completed work for clarity, consistency, and debugging.

6 References

Karpathy’s micrograd (the small-autograd-engine design pattern this Tensor class follows); Goodfellow, Bengio & Courville, *Deep Learning*, Ch. 6 (backpropagation), Ch. 7 (regularization), Ch. 8 (optimization); Kingma & Ba, “Adam: A Method for Stochastic Optimization” (2015); course Weeks 1–2 lecture notes.